Supplementary Topics on Testing

Minsoo Ryu

Hanyang University

Topics Covered

1. Test-Driven Development
2. Test Coverage
3. Risk-based Testing
4. Mutation Analysis
5. Automated Testing

What is TDD?

- Test-driven development (TDD) is a software development process that relies on the repetition of a very short development cycle:
  - first the developer writes a failing automated test case that defines a desired improvement or new function,
  - then produces code to pass that test and finally refactors the new code to acceptable standards

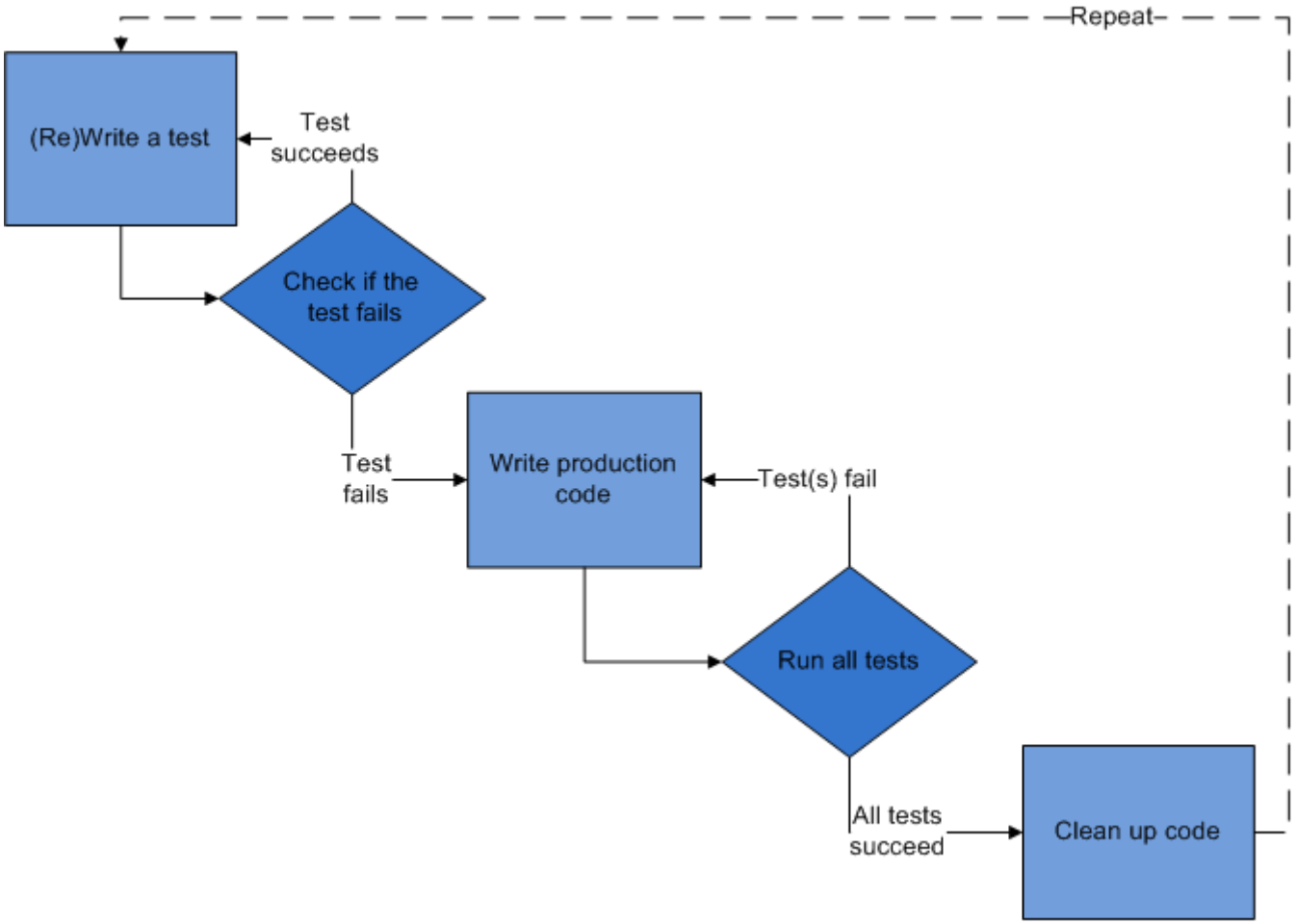
- Kent Beck, who is credited with having developed or 'rediscovered' the technique, stated in 2003 that TDD encourages simple designs and inspires confidence
  - Test-driven development is related to the test-first programming concepts of extreme programming, begun in 1999, but more recently has created more general interest in its own right

Automated Unit Tests

- Test-driven development requires developers to create automated unit tests that define code requirements (immediately) before writing the code itself
  - The tests contain assertions that are either true or false
  - Passing the tests confirms correct behavior as developers evolve and refactor the code

- Developers often use testing frameworks, such as xUnit, to create and automatically run sets of test cases

Test-Driven Development Cycle



Writing a Test

- In test-driven development, each new feature begins with writing a test
  - This test must inevitably fail because it is written before the feature has been implemented
  - If it does not fail, then either the proposed “new” feature already exists or the test is defective

- To write a test, the developer must clearly understand the feature's specification and requirements
  - The developer can accomplish this through use cases and user stories that cover the requirements and exception conditions
  - This could also imply a variant, or modification of an existing test
  - This is a differentiating feature of test-driven development versus writing unit tests after the code is written: it makes the developer focus on the requirements before writing the code, a subtle but important difference

Topics Covered

1. Test-Driven Development
2. Test Coverage
3. Risk-based Testing
4. Mutation Analysis
5. Automated Testing

Test Coverage

- Coverage refers to the extent to which a given verification activity has satisfied its objectives: in essence, providing an exit criteria for when to stop
  - That is, what is "enough" is defined in terms of coverage
- Coverage is a measure, not a method or a test
  - As a measure, coverage is usually expressed as the percentage of an activity that is accomplished
- Two types of test coverage
  - Requirements test coverage
  - Structural test coverage

Requirements Test Coverage

- Software requirements should contain a finite list of behaviors and features, and each requirement should be written to be verifiable
- Testing based on requirements is appealing because it is done from the perspective of the user (thus providing a demonstration of intended function), and allows for development of test plans and cases concurrently with development of the requirements
- Given a finite list of requirements and a set of completion criteria, requirements-based testing becomes a feasible process, unlike exhaustive testing

Requirements Test Coverage

- Unfortunately, a test set that meets requirements coverage is not necessarily a thorough test of the software, for several reasons:
  - the software requirements and the design description (used as the basis for the test set) may not contain a complete and accurate specification of all the behavior represented in the executable code;
  - the software requirements may not be written with sufficient granularity to assure that all the functional behaviors implemented in the source code are tested; and
  - requirements-based testing alone cannot confirm that the code does not include unintended functionality

Structural Test Coverage

- The purpose of structural coverage analysis with the associated structural coverage analysis resolution is to complement requirements-based testing as follows:
  - Provide evidence that the code structure was verified to the degree required for the applicable software level;
  - Provide a means to support demonstration of absence of unintended functions;
  - Establish the thoroughness of requirements-based testing

Types of Structural Coverage

- **Data flow criteria** measure the flow of data between variable assignments and references to the variables
  - Data flow metrics, such as **all-definitions** and **all-uses**, involve analysis of the paths (or subpaths) between the definition of a variable and its subsequent use
  - The structural coverage criteria in many standards, including DO-178B, do not include explicit data flow criteria, but focus on control flow

- **Control flow criteria** measure the flow of control between statements and sequences of statements
  - For control flow criteria, the degree of structural coverage achieved is measured in terms of **statement invocations**, **Boolean expressions evaluated**, and **control constructs exercised**

Statement Coverage

- To achieve statement coverage, every executable statement in the program is invoked at least once during software testing

- Example:

```
if (x > 1) && (y == 0)
    z = z / x;
if (z == 2) || (y > 1)
    z = z + 1;
```

- By choosing x = 2, y = 0, and z = 4 as input to this code segment, every statement is executed at least once
  - However, if an or is coded by mistake in the first statement instead of an and, the test case will not detect a problem

Example

```
Prints (int a, int b) {                          ------------  Printsum is a function
    int result = a+ b;
    If (result> 0)
        Print ("Positive", result)
    Else
        Print ("Negative", result)
    }                                           -----------   End of the source code
```

test case: A = 3, B = 9

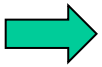
```
1  Prints (int a, int b) {
2  int result = a+ b;
3  If (result> 0)
4      Print ("Positive", result)
5  Else
6      Print ("Negative", result)
7  }
```

statement coverage: 5/7 = 71%

test case: A = -3, B = -9

```
1  Prints (int a, int b) {
2  int result = a+ b;
3  If (result> 0)
4      Print ("Positive", result)
5  Else
6      Print ("Negative", result)
7  }
```

statement coverage: 6/7 = 85%



Statement coverage of test set {(A=3, B=9), (A=-3, B=-9)} is 100%

Decision Coverage

- Decision coverage requires two test cases for each decision: one for a true outcome and another for a false outcome
  - Also known as branch coverage or edge coverage

- For simple decisions (i.e., decisions with a single condition), decision coverage ensures complete testing of control constructs
  - But, not all decisions are simple
  - For the decision (A or B), test cases (TF) and (FF) will toggle the decision outcome between true and false
  - However, the effect of B is not tested; that is, those test cases cannot distinguish between the decision (A or B) and the decision A

Example

```
Demo(int a) {
     If (a> 5)
        a=a*3
     Print (a)
    }
```

test case: a = 2

```
Demo(int a) {
    If (a> 5)
       a=a*3
     Print (a)
    }
```

false

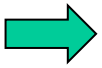
decision coverage: 1/2 = 50%

test case: a = 6

```
Demo(int a) {
    If (a> 5)
       a=a*3
     Print (a)
    }
```

true

decision coverage: 1/2 = 50%



Decision coverage of test set {(a=3), (a=6)} is 100%

Condition Coverage

- Condition coverage requires that each condition in a decision take on all possible outcomes at least once (to overcome the problem in the previous example), but does not require that the decision take on all possible outcomes at least once

- For example, for the decision (A or B) test cases (TF) and (FT) meet the coverage criterion, but do not cause the decision to take on all possible outcomes

- As with decision coverage, a minimum of two test cases is required for each decision

Condition/Decision Coverage (C/DC)

- Condition/decision coverage combines the requirements for decision coverage with those for condition coverage
  - That is, there must be sufficient test cases to toggle the decision outcome between true and false and to toggle each condition value between true and false
  - Hence, a minimum of two test cases are necessary for each decision

- Using the example (A or B), test cases (TT) and (FF) would meet the coverage requirement
  - However, these two tests do not distinguish the correct expression (A or B) from the expression A or from the expression B or from the expression (A and B)

Modified Condition/Decision Coverage (MC/DC)

- The MC/DC criterion enhances the condition/decision coverage criterion by requiring that each condition be shown to independently affect the outcome of the decision
- The independence requirement ensures that the effect of each condition is tested relative to the other conditions
- However, achieving MC/DC requires more thoughtful selection of the test cases and, in general, a minimum of n+1 test cases for a decision with n inputs
  - For the example (A or B), test cases (TF), (FT), and (FF) provide MC/DC
  - For decisions with a large number of inputs, MC/DC requires considerably more test cases than any of the coverage measures discussed above

Multiple Condition Coverage (MCC)

- Finally, multiple condition coverage requires test cases that ensure each possible combination of inputs to a decision is executed at least once; that is, multiple condition coverage requires exhaustive testing of the input combinations to a decision

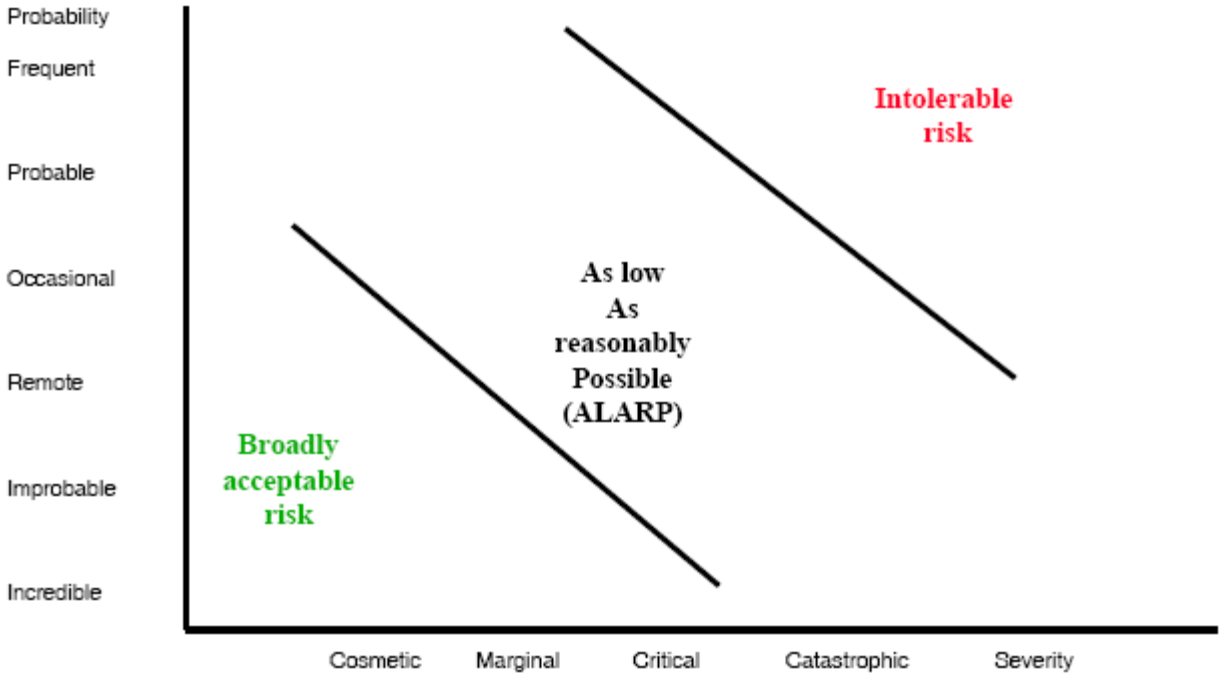
- In theory, multiple condition coverage is the most desirable structural coverage measure; but, it is impractical for many cases
  - For a decision with n inputs, multiple condition coverage requires 2n tests

Topics Covered

1. Test-Driven Development
2. Test Coverage
3. Risk-based Testing
4. Mutation Analysis
5. Automated Testing

What is Risk?

- A factor that could result in a future negative consequence; usually expressed as **impact** and **likelihood** (ISTQB Glossary)



Why Risk-Based Testing?

❒ Why Risk-Based Testing?

- Because there is never enough time to test everything
- Testing often happens at the end of the project — the "caboose effect"
- Therefore, compromises in testing are inevitable

❒ But testing must still find the important defects

- Important defects are those with high risk
- High impact if they occur
- High likelihood of occurring

Risk-Based Testing Process

- Identify risks
  - What the risks are
  - Where they are
- Analyze the risks
  - Impact
  - Likelihood
- Build and perform tests
  - Based on the risk analysis
- Monitor and report

Risk Identification

- Risk identification involves collecting information about the project and classifying it to determine the amount of potential risk
- The risk could be related to system complexity, new technology or methodology involved that could cause problems, limited business knowledge or poor design and code quality
  - Brainstorming is an effective way for risk identification
  - Maximum number of appr. 35 risk items

Risk item 1	Functionality
Risk item 2	Security
Risk item 3	Functionality
Risk item 4	Interoperability

Risk Analysis

- Risk = impact x likelihood
  - What is the impact for the business?
  - What is the likelihood that there are defects?

- Determine factors based on previous projects, e.g. defect patterns

Risk Analysis

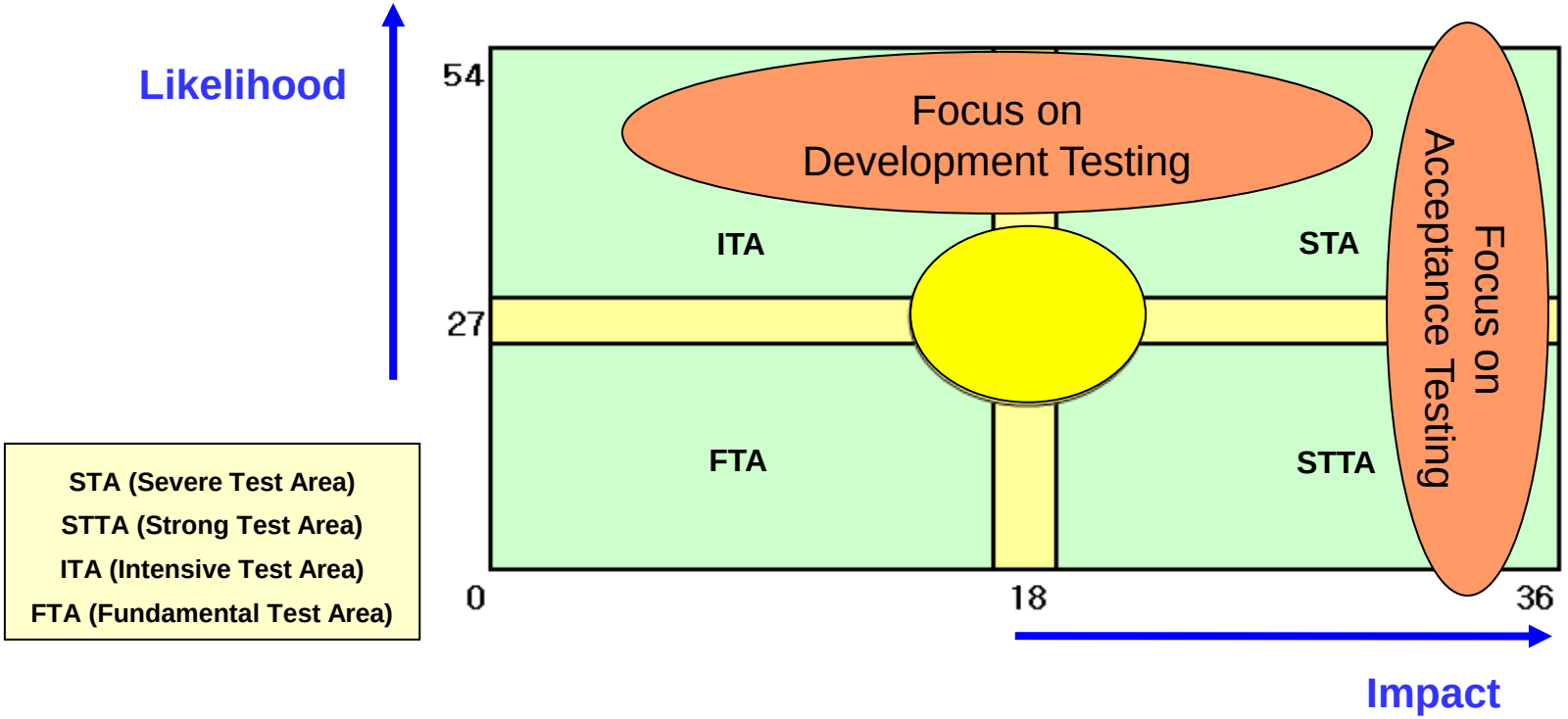
- Likelihood
  - Complexity
  - New development (level of re-uses)
  - Interrelationship (# interfaces)
  - Size
  - Technology
  - Geographical spread
  - Inexperience (of development team)
- Impact
  - user importance (“selling item”)
  - financial (or other)
  - damage (e.g. safety)
  - usage intensity
  - external visibility
  - cost of rework

Stakeholder Involvement

- Stakeholder identification
  - Internal (likelihood) and external (impact)
- Individual stakeholder scoring
- Consensus meeting

	Likelihood						Impact			
	Complexity	New development	Interfacing	Technology	Experience level		Business import.	Usage intensity	Safety	
Item 1	5	3	2	1	5	16	5	4	1	10
Item 2	2	1	2	1	2	8	3	3	1	7
Item n										

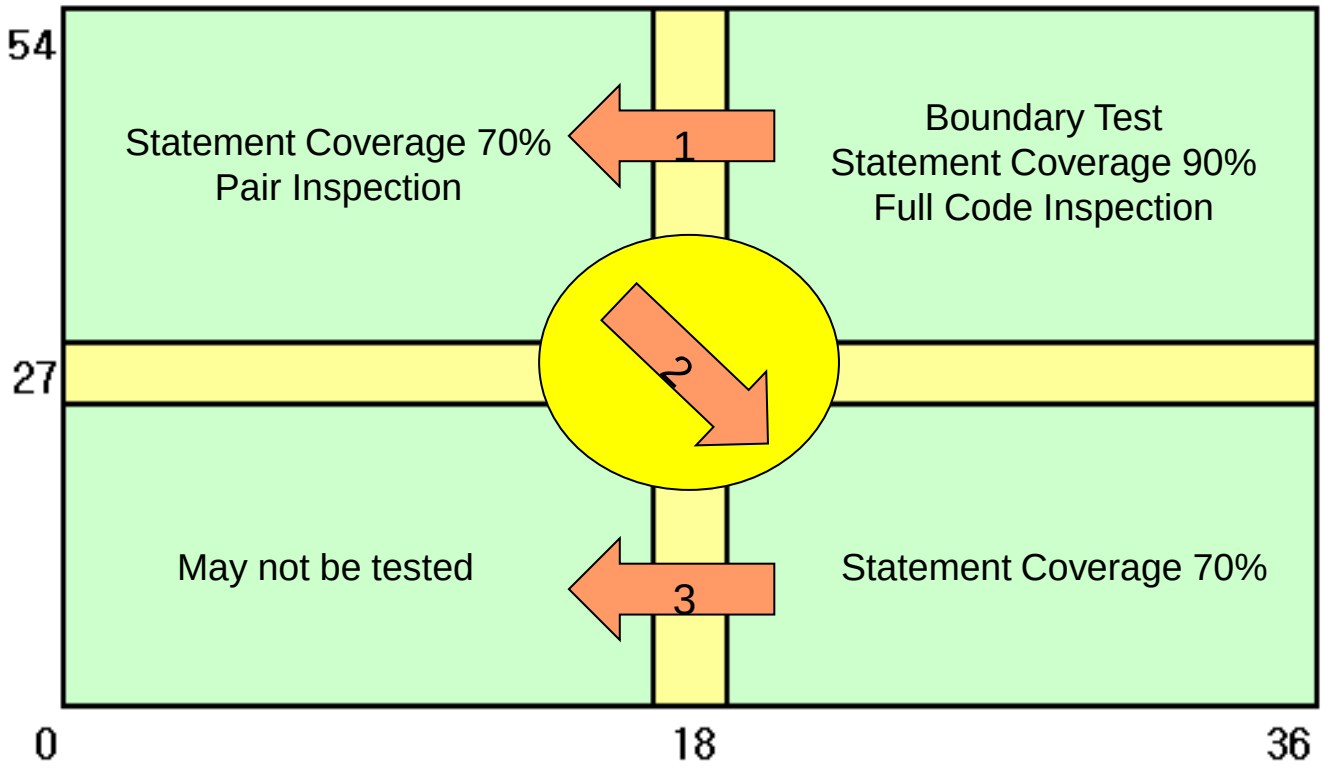
Risk Matrix



STA (Severe Test Area)
STTA (Strong Test Area)
ITA (Intensive Test Area)
FTA (Fundamental Test Area)

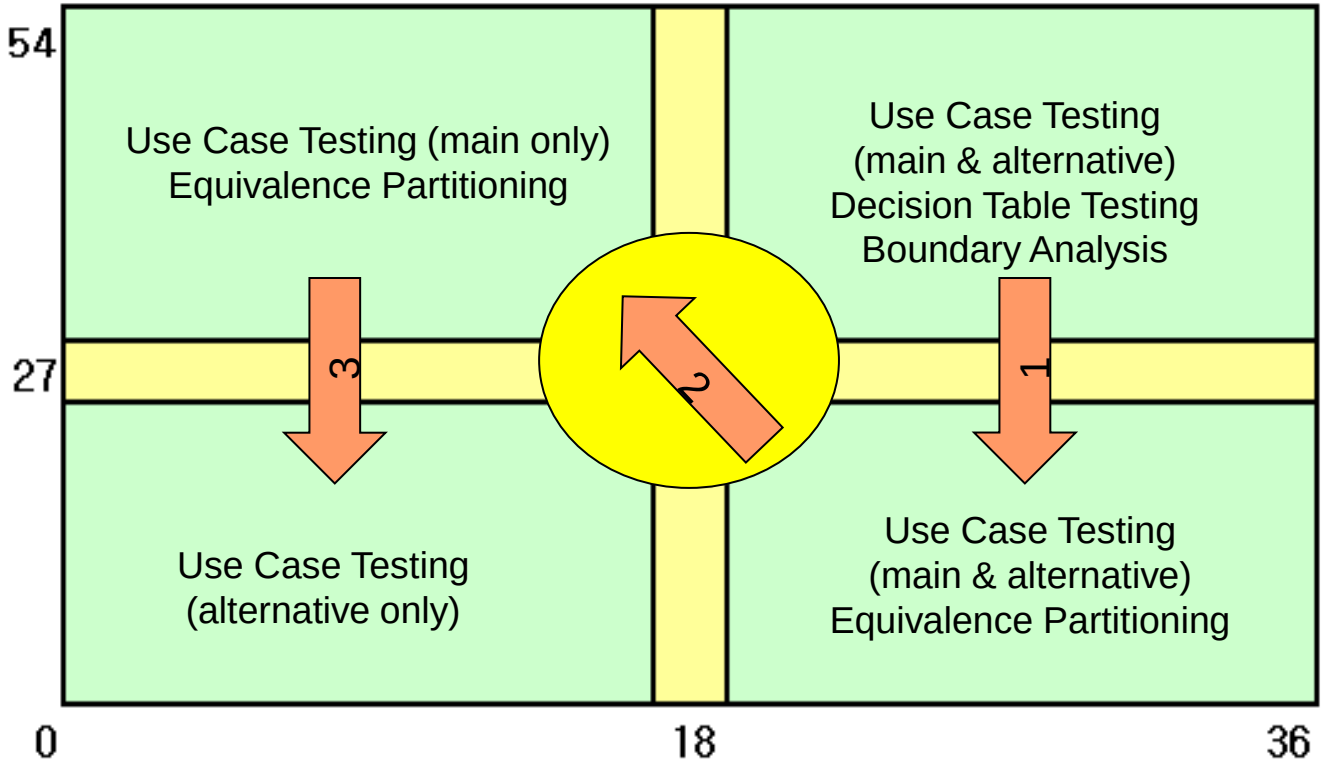
Test Flow Example 1

❒ Focus on Development Testing



Test Flow Example 2

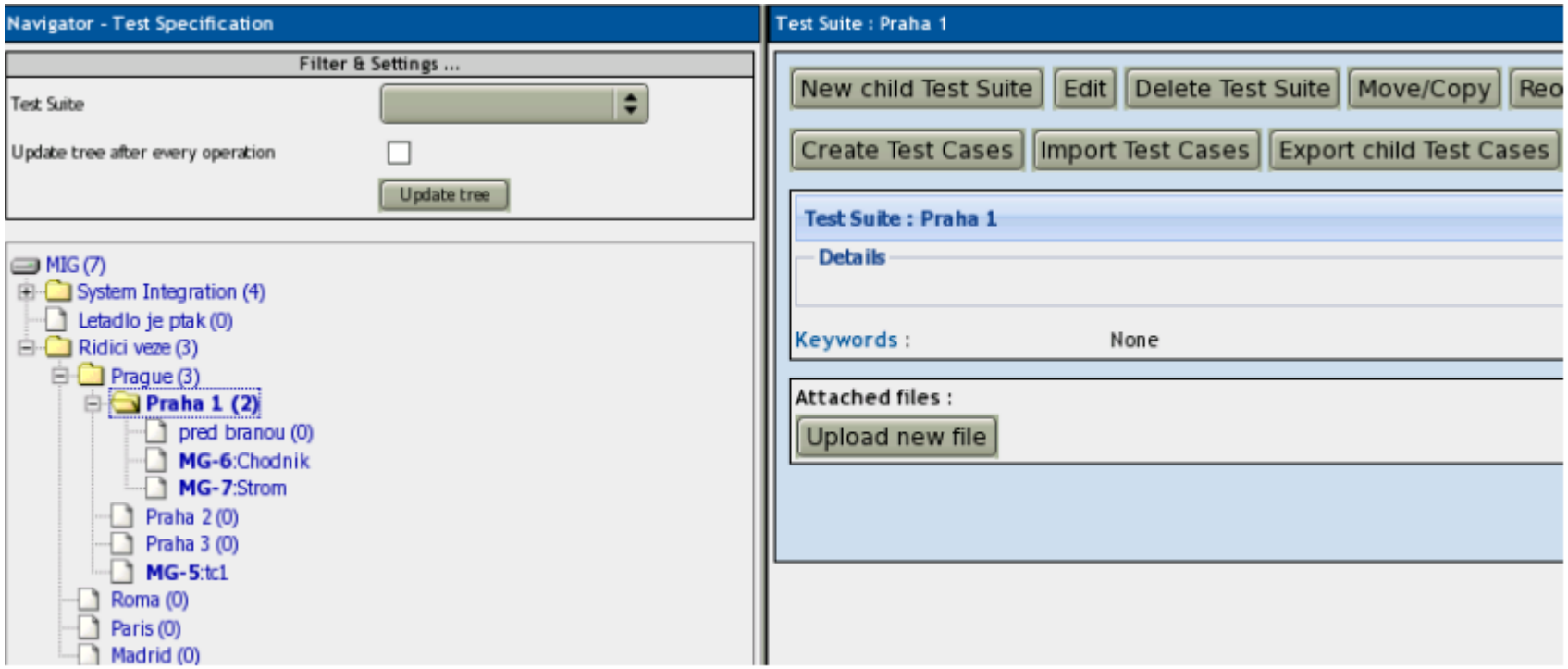
❒ Focus on System/Acceptance Testing




Q & A

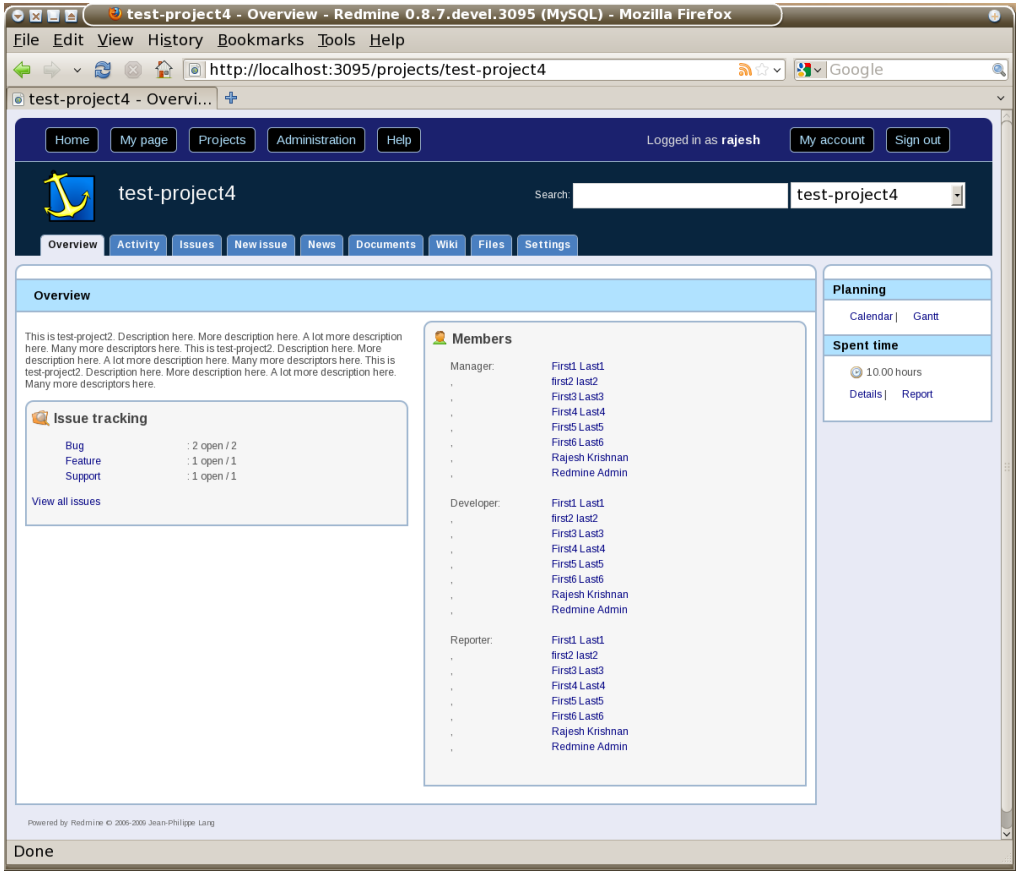
TestLink

❒ Test Case Management



Redmine

- Project management web application
  - Test Scheduling and Management



Bugzilla

❒ Bug tracking system

